

TelemetriaMqtt v2 — Checkpoint 4

Script de ingestão: de mensagens MQTT a pontos no InfluxDB

Projeto de Telemetria FSAE

16 de setembro de 2026

Abstract

Este documento detalha o quarto checkpoint do projeto de reconstrução do sistema de telemetria da equipe: o serviço responsável por assinar o tópico MQTT de telemetria e persistir cada mensagem como um ponto no InfluxDB, iniciando o caminho de armazenamento histórico da arquitetura. O checkpoint aborda uma decisão de design não-trivial sobre qual relógio usar como timestamp da série temporal, e mantém o padrão de testar cada camada com o nível de realismo possível dado o que já existe de infraestrutura.

Contents

1	Contexto e objetivo	1
2	O broker já existe: uma integração real foi possível	2
3	Decisão de design: qual relógio vira o timestamp	2
4	Estrutura de arquivos criada	2
4.1	transform.py	3
4.2	ingest.py	3
4.3	influx_writer.py	4
4.4	__main__.py	4
5	Testes automatizados	4
5.1	transform.py — 5 testes, lógica pura	4
5.2	ingest.py — 3 testes, wiring com writer falso	5
5.3	influx_writer.py — 3 testes, SDK mockado	5
5.4	Resultado da execução	5
6	Validação manual ponta a ponta	6
7	Resumo do que ficou pronto	6
8	Próximos passos	7

1 Contexto e objetivo

A arquitetura do projeto (definida em `SCOPE.md`) separa dois caminhos de consumo dos dados de telemetria: um caminho **ao vivo**, que lê diretamente do broker MQTT via Grafana Live (sem depender de banco de dados), e um caminho **histórico**, que persiste cada mensagem no InfluxDB

para análise posterior — comparar treinos, estudar tendências ao longo de uma temporada, etc. Este checkpoint implementa o serviço que alimenta o segundo caminho.

Uma particularidade da ordem dos checkpoints (documentada em `CLAUDE.md`) merece nota: o script de ingestão (checkpoint 4) é construído **antes** do InfluxDB estar de fato no ar como serviço (checkpoint 5). Isso repete o mesmo padrão já usado no checkpoint 2 (o script mock de publicação foi escrito e testado antes de existir um broker MQTT real): a lógica de negócio é implementada e testada com um cliente do banco de dados *simulado* (*mock*), e a validação contra uma instância real de banco fica reservada para quando essa infraestrutura entrar no ar.

2 O broker já existe: uma integração real foi possível

Diferente do checkpoint 2, neste checkpoint já existia uma peça real de infraestrutura disponível: o broker MQTT, colocado no ar no checkpoint 3. Isso permitiu uma validação manual mais completa do que seria possível antes — o lado de *consumo* de mensagens (conexão MQTT, autenticação, assinatura do tópico, parsing da mensagem) foi testado contra um broker de verdade, publicado por um script mock real (checkpoint 2). Só o lado de *escrita* no InfluxDB permanece simulado neste checkpoint, e será validado com uma instância real no checkpoint seguinte.

3 Decisão de design: qual relógio vira o timestamp

Um ponto no InfluxDB precisa de um timestamp. A opção óbvia — e errada — seria usar diretamente o campo `ts` presente em cada mensagem publicada pelo dispositivo (ESP32 real ou o mock publisher). O problema: esse `ts` **não é hora real**. No firmware do ESP32, ele viria da função `millis()`, que conta milissegundos desde o boot do dispositivo — reinicia do zero toda vez que o ESP32 liga. No mock publisher, é o tempo decorrido desde o início do próprio script. Em nenhum dos dois casos esse número corresponde a uma data e hora do calendário.

Se esse valor fosse usado como timestamp do banco, todo o histórico apareceria com datas erradas (por volta de 1970, já que `ts` costuma começar próximo de zero), e reinícios do dispositivo causariam saltos incoerentes na linha do tempo dos dados.

A solução adotada: o **timestamp do ponto gravado no InfluxDB é o horário em que a mensagem chegou** ao serviço de ingestão — o relógio de parede (*wall clock*) do próprio servidor onde a ingestão roda, não o relógio do dispositivo remoto. O valor original de `ts` não é descartado: ele é preservado como um campo comum do ponto (`device_ts_ms`), disponível para análises futuras que precisem dele — por exemplo, calcular a latência entre a geração do dado no dispositivo e sua chegada ao servidor, ou detectar mensagens fora de ordem.

4 Estrutura de arquivos criada

Listing 1: Estrutura do diretório backend/ingestion/

```
1 backend/
2   ingestion/
3     __init__.py
4     transform.py      # lógica pura: payload -> InfluxPoint
5     ingest.py         # callback MQTT: parseia e chama o writer
6     influx_writer.py  # wiring com o SDK oficial do InfluxDB
7     __main__.py       # ponto de entrada de linha de comando
8   tests/
9     test_ingestion_transform.py
10    test_ingest.py
11    test_influx_writer.py
```

A mesma separação de responsabilidades usada no checkpoint 2 foi repetida aqui: **lógica pura** (transformação de dados, sem nenhum I/O) separada da **integração** (rede, banco de dados) — o que torna a lógica de transformação trivialmente testável, e permite testar a integração com dependências simuladas.

4.1 transform.py

Listing 2: Trecho de ingestion/transform.py

```
1 MEASUREMENT = "telemetry"
2
3 NUMERIC_FIELDS = [
4     "rpm", "throttlePosition", "lambda", "map", ...
5 ]
6
7 @dataclass
8 class InfluxPoint:
9     measurement: str
10    fields: dict[str, Any]
11    time_ns: int
12
13 def to_point(payload: dict, received_at_ns: int) -> InfluxPoint:
14     missing = [name for name in NUMERIC_FIELDS if name not in payload]
15     if missing:
16         raise ValueError(f"payload sem os campos: {' ', ' '.join(missing)}")
17
18     fields = {name: payload[name] for name in NUMERIC_FIELDS}
19     fields["device_ts_ms"] = payload.get("ts")
20
21     return InfluxPoint(measurement=MEASUREMENT, fields=fields,
22                        time_ns=received_at_ns)
```

`InfluxPoint` é um tipo próprio do projeto, sem nenhuma dependência do SDK oficial do InfluxDB — a tradução para o tipo `Point` da biblioteca `influxdb-client` só acontece na camada de integração (`influx_writer.py`). Essa indireção mantém a lógica de transformação testável sem precisar importar ou simular a biblioteca do banco de dados.

Mensagens com campos faltando levantam um `ValueError` explícito, em vez de gravar um ponto incompleto silenciosamente — coerente com a postura do projeto de preferir falhar de forma visível a persistir dado corrompido.

4.2 ingest.py

Listing 3: Trecho de ingestion/ingest.py

```
1 def make_on_message(writer, clock=time.time_ns):
2     def on_message(client, userdata, msg):
3         try:
4             payload = json.loads(msg.payload)
5             point = to_point(payload, received_at_ns=clock())
6             except (json.JSONDecodeError, ValueError) as exc:
7                 logger.warning("mensagem descartada: %s", exc)
8                 return
9             writer(point)
10    return on_message
```

`make_on_message` é uma função de ordem superior: recebe um `writer` (qualquer função capaz de "consumir" um `InfluxPoint`) e devolve o callback `on_message` no formato exato que a biblioteca `paho-mqtt` espera. Essa indireção permite trocar o destino real (gravação no InfluxDB) por um destino de teste (uma lista Python, por exemplo) sem duplicar nenhuma lógica de parsing.

Qualquer exceção de parsing (JSON malformatado) ou de validação (campo faltando, levantada por `to_point`) é capturada e logada, sem propagar — uma mensagem ruim nunca derruba o processo de ingestão inteiro, que precisa continuar rodando de forma contínua durante um treino ou corrida.

4.3 `influx_writer.py`

Listing 4: `ingestion/influx_writer.py`

```
1 def make_influx_writer(write_api, bucket: str, org: str):
2     def write(point: InfluxPoint) -> None:
3         influx_point = Point(point.measurement).time(
4             point.time_ns, WritePrecision.NS)
5         for name, value in point.fields.items():
6             if value is not None:
7                 influx_point = influx_point.field(name, value)
8         write_api.write(bucket=bucket, org=org, record=influx_point)
9     return write
```

Aqui, e só aqui, o `InfluxPoint` interno do projeto é traduzido para o tipo `Point` do SDK oficial (`influxdb-client`), e de fato enviado para o banco através do `write_api` passado por parâmetro. Campos com valor `None` (por exemplo, `device_ts_ms` quando o payload não trouxe `ts`) são pulados — o próprio SDK do InfluxDB não aceita gravar um campo nulo.

4.4 `__main__.py`

O ponto de entrada de linha de comando (`python -m ingestion`) conecta as três peças: assina o tópico MQTT como o usuário `telemetria_reader` (definido no checkpoint 3, com permissão só de leitura), e escreve através de um `writer` real (`influxdb-client`, conectado via `-influx-url/-influx-token/-influx-org/-influx-bucket`) ou, em modo `-dry-run`, um `writer` que apenas imprime cada ponto no terminal — permitindo rodar e observar o serviço de ponta a ponta mesmo sem o InfluxDB ainda existir.

Um detalhe da implementação: a biblioteca `influxdb-client` só é importada dentro da função que constrói o `writer` real, não no topo do arquivo — assim, rodar em modo `-dry-run` nem exige essa dependência estar resolvida, evitando qualquer acoplamento desnecessário nesse modo de uso.

5 Testes automatizados

Onze testes novos, escritos antes ou junto de cada função correspondente, mantendo a granularidade das camadas descritas na Seção anterior.

5.1 `transform.py` — 5 testes, lógica pura

- Teste 1** `test_to_point_maps_all_20_numeric_fields` — confirma que todo sinal declarado vira um campo do ponto, nenhum esquecido.
- Teste 2** `test_to_point_uses_measurement_name` — confirma o nome fixo da `measurement` ("telemetry").
- Teste 3** `test_to_point_uses_received_at_as_time_not_device_ts` — valida diretamente a decisão de design da Seção 3: o `time_ns` do ponto é o horário de chegada passado como parâmetro, não derivado do `ts` do payload.
- Teste 4** `test_to_point_keeps_device_ts_as_a_reference_field` — confirma que o `ts` original não é perdido, aparecendo como `device_ts_ms`.

Teste 5 `test_to_point_raises_on_missing_field` — confirma que um payload incompleto levanta `ValueError` em vez de ser aceito parcialmente.

5.2 `ingest.py` — 3 testes, wiring com writer falso

Teste 1 `test_on_message_parses_valid_payload_and_calls_writer` — simula uma mensagem MQTT válida chegando e confirma que o writer é chamado com o ponto correto.

Teste 2 `test_on_message_ignores_invalid_json` — confirma que uma mensagem com JSON corrompido é descartada silenciosamente (do ponto de vista do writer, que nunca é chamado).

Teste 3 `test_on_message_ignores_payload_missing_fields` — mesmo comportamento, para um payload estruturalmente válido mas incompleto.

5.3 `influx_writer.py` — 3 testes, SDK mockado

Teste 1 `test_writer_calls_write_api_with_correct_bucket_and_org` — confirma que o `write_api.write(...)` é chamado com o bucket e a organização configurados.

Teste 2 `test_writer_builds_point_with_measurement_fields_and_time` — inspeciona o *line protocol* do ponto construído (formato textual que o InfluxDB usa internamente) para confirmar measurement, campos e timestamp corretos, incluindo a formatação de inteiros (sufixo `i`) do próprio protocolo do banco.

Teste 3 `test_writer_skips_none_fields` — confirma que um campo com valor `None` não é enviado ao SDK (que rejeitaria essa chamada).

5.4 Resultado da execução

Listing 5: Saída real de `pytest -v` (suíte completa do backend/)

```
1 tests/test_influx_writer.py::
  test_writer_calls_write_api_with_correct_bucket_and_org PASSED
2 tests/test_influx_writer.py::
  test_writer_builds_point_with_measurement_fields_and_time PASSED
3 tests/test_influx_writer.py::test_writer_skips_none_fields PASSED
4 tests/test_ingest.py::test_on_message_parses_valid_payload_and_calls_writer
  PASSED
5 tests/test_ingest.py::test_on_message_ignores_invalid_json PASSED
6 tests/test_ingest.py::test_on_message_ignores_payload_missing_fields PASSED
7 tests/test_ingestion_transform.py::test_to_point_maps_all_20_numeric_fields
  PASSED
8 tests/test_ingestion_transform.py::test_to_point_uses_measurement_name PASSED
9 tests/test_ingestion_transform.py::
  test_to_point_uses_received_at_as_time_not_device_ts PASSED
10 tests/test_ingestion_transform.py::
  test_to_point_keeps_device_ts_as_a_reference_field PASSED
11 tests/test_ingestion_transform.py::test_to_point_raises_on_missing_field
  PASSED
12 [... 11 testes do checkpoint 2, intactos ...]
13
14 22 passed in 0.15s
```

Os 11 testes dos checkpoints anteriores continuam passando sem nenhuma alteração — consistente com a regra de processo estabelecida no checkpoint 2 (testes já escritos nunca são modificados).

6 Validação manual ponta a ponta

Com o broker MQTT real (checkpoint 3) disponível, foi possível validar boa parte do fluxo com componentes reais, não apenas simulados: o script mock (checkpoint 2) publicando de verdade, através do broker de verdade, sendo consumido pelo script de ingestão novo, em modo `-dry-run` (já que o InfluxDB ainda não existe):

Listing 6: Comando de validação manual

```
1 docker compose up -d # sobe o broker real (infra/)
2
3 python -m ingestion --host 127.0.0.1 --port 1883 --no-tls \
4     --username telemetria_reader --password <senha-local-de-dev> --dry-run &
5
6 python -m mock_publisher --host 127.0.0.1 --port 1883 --no-tls \
7     --username esp32_telemetria --password <senha-local-de-dev> --duration 1.5
```

Listing 7: Trecho da saída real observada

```
1 Assinando 'telemetria/esp32/data' em 127.0.0.1:1883, gravando em (dry-run)
2 [dry-run] telemetry {'rpm': 4400.0, ..., 'device_ts_ms': 0} t
   =1789579648744539691
3 [dry-run] telemetry {'rpm': 4588.41, ..., 'device_ts_ms': 50} t
   =1789579648794445042
4 [dry-run] telemetry {'rpm': 4776.3, ..., 'device_ts_ms': 100} t
   =1789579648844634876
```

O resultado confirma visualmente a decisão da Seção 3: o campo `device_ts_ms` mostra os valores relativos originais do mock (0, 50, 100, ...), enquanto `t` (o `time_ns` de fato usado como timestamp do ponto) mostra um valor de relógio real, em nanossegundos desde a época Unix — exatamente a separação pretendida entre os dois conceitos de tempo.

Detalhe encontrado durante a execução

A primeira tentativa dessa validação não mostrou nenhuma saída do script de ingestão, mesmo funcionando corretamente por trás dos panos. A causa: o comando usado para limitar a duração da execução (`timeout`) envia o sinal `SIGTERM` ao processo Python, que por padrão **não** garante o descarregamento (*flush*) do buffer de saída padrão antes de encerrar — as linhas impressas ficaram retidas em memória e nunca chegaram ao terminal. A correção foi rodar o Python em modo não-bufferizado (`python -u`), que escreve cada linha imediatamente. Esse é um lembrete de que `print()` para depuração em processos de longa duração deve levar em conta como o processo é efetivamente encerrado em produção.

7 Resumo do que ficou pronto

- Serviço de ingestão completo, assinando o broker como usuário read-only e transformando mensagens em pontos do InfluxDB.
- Decisão de design documentada e testada: timestamp do banco é o horário de recebimento, não o relógio (não-absoluto) do dispositivo remoto.
- Tratamento explícito de mensagens inválidas, sem gravação parcial e sem derrubar o processo.
- 11 testes automatizados novos, mantendo a separação entre lógica pura e integração já estabelecida no checkpoint 2.

- Validação manual real do lado de consumo MQTT (broker de verdade), com o lado de escrita no InfluxDB simulado até a próxima etapa.

8 Próximos passos

O checkpoint 5 coloca o InfluxDB real no ar (via Docker, seguindo o mesmo padrão do broker MQTT), permitindo validar pela primeira vez a escrita de ponta a ponta contra uma instância de banco de dados de verdade — e confirmar definitivamente a suposição de InfluxDB 2.x levantada neste checkpoint.